

FASE 1 — Modelo de datos y seguridad (RLS)

Alcance: extensiones **aditivas** al esquema real de BuscoEdu (documentado en `BUS-COEDU_DATA_DICTIONARY`) para habilitar el Lead Center, más el modelo de seguridad por filas (RLS) para los roles `super_admin` y `asesor`. **No se elimina ni renombra nada**, no se crean tablas paralelas a las existentes del CRM, y no hay borrado físico en ningún punto.

1. Migraciones creadas

Todas viven en `supabase/migrations/` y son **idempotentes** (`IF NOT EXISTS`, `DROP POLICY IF EXISTS` + `CREATE`, `DO $$...$$` con guardas). Pueden re-ejecutarse sin efectos colaterales.

Orden	Archivo	Propósito
1	<code>20260829120000_leadcenter_extension_personas_identidad.sql</code>	Identidad por celular E.164 en <code>personas</code>
2	<code>20260829120100_leadcenter_desafios_otp.sql</code>	Tabla NUEVA <code>desafios_otp</code> (capacidad inexistente)
3	<code>20260829120200_leadcenter_modelo_negocio.sql</code>	<code>modelo_negocio</code> en oferta + snapshot/idempotencia en oportunidad
4	<code>20260829120300_leadcenter_rules_crm.sql</code>	Funciones de identidad + RLS del CRM
5	<code>20260829120400_leadcenter_seeds.sql</code>	Seeds: rol <code>asesor</code> , tipos de consentimiento, etapas, subestados, reglas
6	<code>20260829120500_leadcenter_fn_convertir_aplicacion.sql</code>	RPC transaccional de conversión (detallado en Fase 3)

1.1 Extensión de `personas` (migración 1)

Se agregan columnas, **sin tocar** `telefono_principal` / `whatsapp` / `correo_principal` existentes:

- `celular_e164` TEXT — celular normalizado E.164, **llave funcional** para reconciliar identidad entre canales. No reemplaza `personas.id` (PK).
- `pais_celular` TEXT — código de país usado para normalizar (trazabilidad).
- `auth_user_id` UUID → FK `auth.users(id)` ON DELETE SET NULL — sesión self-service de la persona.
- `telefono_verificado`, `whatsapp_verificado` BOOLEAN DEFAULT false.
- `metodo_verificacion` TEXT — `simulated` | `twilio` | `whatsapp`.
- `fecha_verificacion_celular` TIMESTAMPTZ.

Índices: único parcial en `celular_e164` (permite NULL), único parcial en `auth_user_id`, y de búsqueda en `correo_principal`, `visitante_id`, `estado_relacion`.

No se hace backfill automático de `telefono_principal` → `celular_e164`: la reconciliación se hace en servidor (Fase 2). Si al normalizar aparecieran colisiones, se reportan como incidencia; **nunca se fusionan** registros automáticamente.

1.2 Tabla `desafios_otp` (migración 2)

Cubre una capacidad **inexistente** en el esquema (no hay ninguna tabla OTP). El código OTP **jamás** se guarda en claro: solo `codigo_hash` (bcrypt). Campos clave: `celular_e164`, `proposito` (registro/login/reverificacion), `proveedor` (simulated/twilio/whatsapp), `estado`, `intentos`, `max_intentos` DEFAULT 5, `ip_origen`, `visitante_id`, `persona_id`, `expira_en`, `verificado_en`. El proveedor real reutilizará esta misma tabla sin cambios estructurales.

RLS: habilitada **sin** políticas de escritura pública; solo `super_admin` puede hacer SELECT. Toda la operación real ocurre desde el servidor con service role (que bypassa RLS), evitando enumeración de celulares.

1.3 Modelo de negocio (migración 3)

- `ofertas_academicas.modelo_negocio` CHECK ('por_lead', 'por_inscrito'), **backfill conservador** a `por_inscrito` (el flujo que NO transfiere datos).
- `oportunidades.modelo_negocio_snapshot` — congela el modelo al crear la oportunidad, para que un cambio futuro de la oferta no altere el histórico.
- `oportunidades.clave_idempotencia` + índice único parcial — evita oportunidades duplicadas ante doble clic / reintento de red.
- Índices para dashboard/pipeline (`etapa_id`, `subestado_id`, `asesor_asignado_id`, `estado`, `persona_id`, `universidad_id`, `fecha_proxima_accion`, `actualizado_en`).

2. Modelo de seguridad (RLS) — migración 4

2.1 Cadena de identidad

```
auth.uid()
  → usuarios_internos.auth_user_id
  → usuarios_internos.id           (rol vía roles.codigo)
  → oportunidades.asesor_asignado_id (asignación vigente)
```

Funciones SECURITY DEFINER (para poder leer `usuarios_internos` / `roles` sin exponerlas):

- `usuario_interno_id()` → `usuarios_internos.id` del usuario activo (o NULL).
- `es_asesor_o_super()` → BOOLEAN (rol `asesor` o `super_admin` activo).
- `puede_ver_oportunidad(uuid)` → super ve todo; asesor solo las asignadas a él.
- `puede_ver_persona(uuid)` → super ve todas; asesor solo personas con al menos una oportunidad asignada.

Se **reutiliza** `is_super_admin()` ya existente en `EJECUTAR_EN_SUPABASE.sql` (no se redefine).

2.2 Políticas

- Solo se **agregan** políticas nombradas con prefijo `lc_` (y `otp_...` para OTP). No se elimina ni reemplaza ninguna política pública existente (lectura de catálogo, INSERT anónimo del explorador en `visitantes` / `eventos_negocio`).

- `oportunidades` : SELECT/UPDATE para super (todo) y asesor (asignadas).
- `personas` : SELECT por `puede_ver_persona(id)` .
- Tablas hijas de oportunidad (aplicaciones, propuestas, versiones, historiales, asignaciones, transferencias, comunicaciones, conversaciones, mensajes, hechos NaIA, consentimientos, preferencias, perfil): SELECT acotado por `puede_ver_oportunidad` / `puede_ver_persona` .
- `notas_crm` y `tareas_crm` : SELECT + INSERT (+ UPDATE en tareas) para el asesor asignado / super.
- Catálogos del embudo (`etapas_embudo` , `subestados_oportunidad` , `reglas_estancamiento`): SELECT para cualquier asesor/super.
- `eventos_negocio` : se agrega SELECT para asesor/super **sin tocar** el INSERT anónimo del explorador.

Importante: todas las escrituras del flujo de conversión de una persona anónima se hacen por **RPC con service role** y NO dependen de estas políticas de `authenticated` . Las políticas RLS gobiernan lo que el asesor/super ve y edita desde el Lead Center autenticado.

3. Seeds (migración 5)

Idempotentes, solo insertan si no existe:

- Rol `asesor` .
- 4 tipos de consentimiento: `tratamiento_datos` (obligatorio), `contacto` , `contacto_whatsapp` , `transferencia_universidad` — con `texto_completo` legal en español.
- 6 etapas del embudo (Nuevo, En gestión, Calificada, Propuesta/Transferencia, Ganada, Perdida) — solo si la tabla está vacía.
- Subestados base para Nuevo/En gestión — solo si no hay subestados.
- 1 regla de estancamiento base (Nuevo, 24h → crear tarea + reducir score) — solo si no hay reglas.

4. Cumplimiento de restricciones absolutas

- ☒ Sin claves hardcodeadas (todo por env / service role en servidor).
- ☒ Sin eliminar/renombrar tablas ni columnas.
- ☒ Sin tablas paralelas a personas/oportunidades/propuestas/etapas/historial/tareas/notas/transferencias — solo se **extienden** y se agrega la tabla nueva `desafios_otp` (capacidad inexistente).
- ☒ Sin borrado físico (los flags `activo` / `estado` existentes cubren la baja lógica).
- ☒ Transferencia solo con consentimiento vigente (garantizado por `transferencias_universidad.consentimiento_id NOT NULL` + lógica del RPC en Fase 3).

5. Estado de verificación

- **Escrito y revisado** el SQL; sintaxis validada manualmente contra el diccionario de datos.
- **NO ejecutado** contra Supabase: el entorno de este paso **no dispone de credenciales** (`SUPABASE_*` vacías). Las migraciones quedan **preparadas para aplicar** con `supabase db push` o pegándolas en el SQL Editor, en el orden numérico indicado.
- Verificación recomendada tras aplicar: correr las 3 queries de snapshot del diccionario (columnas, FKs, CHECKs) y confirmar la aparición de las nuevas columnas/tabla/políticas.

6. Riesgos y mitigaciones

Riesgo	Mitigación
Nombres exactos de columnas menores difieren del diccionario	El SQL usa solo FKs/columnas documentadas; el RPC (Fase 3) inserta campos mínimos y deja opcionales anulables. Verificar al aplicar.
RLS pública preexistente creada fuera del repo	Solo se agregan políticas <code>lc_ / otp_</code> ; nunca se hace <code>DROP</code> de políticas ajenas.
Colisión de celular al normalizar históricos	No hay backfill; se resuelve en runtime y se reporta como incidencia.
<code>is_super_admin()</code> no existiera en algún entorno	Está en <code>EJECUTAR_EN_SUPABASE.sql</code> ; documentado como requisito.